

桌面GUI智能体开发需求文档(PRD)

1. 文档信息

项	内容
文档名称	基于多模态大模型的桌面GUI智能体开发需求文档
项目名称	桌面GUI智能体原型系统
版本	v1.1
编制日期	2026-01-09
适用对象	开发工程师

2. 项目概述

2.1 项目背景

当前GUI智能体技术在办公自动化、软件测试、无障碍交互等领域具有广泛应用前景。主流技术路线均采用"多模态视觉理解+大模型任务规划+桌面控制执行"的架构，能够实现对计算机屏幕内容的理解和自动化操作。

本项目旨在基于业界主流开源技术栈，开发一个能够"看懂屏幕、操作电脑"的桌面GUI智能体原型，实现端到端的任务执行能力，为后续拓展应用场景奠定技术基础。

2.2 项目目标

- 实现桌面屏幕内容的实时感知与UI元素识别
- 基于多模态大模型实现自然语言指令理解与任务规划
- 实现鼠标键盘的自动化控制与操作执行
- 构建"感知-思考-执行-反馈"的完整闭环系统
- 完成基础桌面任务的自动化执行，达到预期的成功率与性能指标

2.3 项目范围

包含功能：

- 跨平台屏幕截图与内容识别
- 鼠标键盘基本操作控制
- 基于通义千问多模态模型的任务理解与规划
- 基础桌面任务的端到端执行
- 命令行交互界面

- 基本的错误处理与日志记录

不包含功能：

- 复杂多智能体协作
- 模型微调（可选拓展）
- 图形化交互界面（可选拓展）
- 远程桌面控制
- 商业级安全与权限控制

3. 总体设计

3.1 系统架构

系统采用分层架构设计，分为四层：

交互层
命令行界面 / 结果反馈
Agent核心层
模型调用 / 任务规划 / 动作解析
能力层
桌面感知模块 / 桌面控制模块
基础设施层
Python / PyTorch / 开源库

3.2 核心流程

系统执行任务的核心流程如下：

- 用户通过命令行输入自然语言指令
- 桌面感知模块截取当前屏幕图像
- 将屏幕图像与用户指令一起发送给多模态大模型
- 大模型分析屏幕内容，理解用户指令，生成下一步操作动作
- 桌面控制模块执行生成的操作动作
- 重复步骤2-5，直到任务完成或达到最大重试次数
- 向用户反馈任务执行结果

3.3 技术栈

类别	技术选型
----	------

编程语言	Python 3.10+
深度学习框架	PyTorch 2.2+
多模态大模型	Qwen2-VL-2B-Instruct（默认） / Qwen2-VL-7B-Instruct
Agent框架	AgentScope
屏幕截图	mss
OCR识别	PaddleOCR PP-OCRv4
计算机视觉	OpenCV
鼠标键盘控制	pynput
模型部署	Transformers
数据处理	Pandas、NumPy
单元测试	pytest
版本控制	Git + GitHub

3.4 基础开发环境配置指南

3.4.1 前置条件

- 操作系统：Windows 10/11、macOS 12+、Ubuntu 20.04+
- 硬件要求：推荐配备8GB以上显存的NVIDIA GPU；无GPU可使用在线API模式
- 网络环境：可访问GitHub、Hugging Face、魔搭社区等开源平台

3.4.2 Python与虚拟环境配置

1. 下载并安装Anaconda/Miniconda（推荐）：

<https://docs.conda.io/en/latest/miniconda.html>

2. 打开终端/命令提示符，创建独立虚拟环境：

```
conda create -n gui_agent python=3.10 -y
conda activate gui_agent
```

3. 配置pip国内镜像源（加速下载）：

```
pip config set global.index-url https://pypi.tuna.tsinghu
```

3.4.3 PyTorch与CUDA配置

1. 访问PyTorch官网（<https://pytorch.org/>），根据系统与CUDA版本获取安装命令
2. 推荐安装命令（支持CUDA 12.1）：

```
pip3 install torch torchvision torchaudio --index-url htt
```

3. 验证CUDA是否可用:

```
import torch
print(torch.cuda.is_available()) # 输出True表示配置成功
```

3.4.4 项目依赖安装

创建requirements.txt文件, 写入以下内容:

```
transformers>=4.40.0
accelerate>=0.30.0
bitsandbytes>=0.43.0
agentscope>=0.2.0
mss>=9.0.0
opencv-python>=4.8.0
paddlepaddle>=2.5.0
paddleocr>=2.7.0
pynput>=1.7.6
pandas>=2.0.0
numpy>=1.24.0
pytest>=7.4.0
pillow>=10.0.0
requests>=2.31.0
```

执行安装命令:

```
pip install -r requirements.txt
```

3.4.5 环境验证

创建verify_env.py文件, 运行以下代码验证所有核心依赖:

```
import torch
import cv2
import mss
import paddleocr
from agentscope.models import ModelWrapper

print("PyTorch版本:", torch.__version__)
print("CUDA可用:", torch.cuda.is_available())
print("OpenCV版本:", cv2.__version__)
print("mss版本:", mss.__version__)
print("PaddleOCR可用:", paddleocr.__version__)
print("AgentScope导入成功")
print("所有依赖验证通过！")
```

4. 详细功能需求

4.1 桌面感知模块

4.1.1 屏幕截图功能

- **功能描述：**实现跨平台的屏幕实时截图能力
- **输入参数：**
 - `screen_id`: 可选，指定截图的屏幕编号，默认0
 - `region`: 可选，指定截图区域，格式为(left, top, width, height)，默认全屏
- **输出：**PIL.Image对象
- **业务逻辑：**
 1. 支持Windows、macOS、Linux三大操作系统
 2. 自动适配不同屏幕分辨率与DPI缩放
 3. 单帧截图耗时≤50ms
 4. 支持多屏幕环境下的指定屏幕截图
- **异常处理：**
 - 截图失败时抛出异常并记录日志
 - 处理屏幕不存在、区域越界等错误

4.1.2 OCR文字识别功能

- **功能描述：**识别屏幕截图中的文字内容与位置
- **输入参数：**PIL.Image对象（屏幕截图）
- **输出：**识别结果列表，每个元素包含：
 - `text`: 识别到的文字内容
 - `bbox`: 文字所在区域的边界框坐标，格式为(x1, y1, x2, y2)
 - `confidence`: 识别置信度

- **业务逻辑：**
 1. 集成PaddleOCR PP-OCRv4模型
 2. 自动处理不同分辨率的截图
 3. 支持中英文混合识别
 4. 识别准确率 $\geq 90\%$ （清晰屏幕文字）
- **异常处理：**
 - 模型加载失败时抛出异常
 - 识别结果为空时返回空列表

4.1.3 UI元素定位与可视化功能

- **功能描述：**在截图上标注识别到的UI元素与坐标
- **输入参数：**
 - image：原始屏幕截图
 - elements：识别到的UI元素列表
- **输出：**标注后的PIL.Image对象
- **业务逻辑：**
 1. 使用矩形框标注每个UI元素的边界
 2. 在每个元素附近标注其序号与文本内容
 3. 不同类型元素使用不同颜色标注
- **异常处理：**
 - 输入为空时返回原始图像

4.2 桌面控制模块

4.2.1 鼠标操作功能

- **功能描述：**实现鼠标的基本操作控制
- **支持操作：**
 1. 移动鼠标到指定坐标：move_to(x, y)
 2. 左键点击：click(x=None, y=None, button='left')
 3. 右键点击：right_click(x=None, y=None)
 4. 双击：double_click(x=None, y=None)
 5. 拖拽：drag_from_to(x1, y1, x2, y2, duration=0.5)
- **输入参数：**操作类型与对应坐标
- **输出：**无
- **业务逻辑：**
 1. 坐标基于屏幕左上角原点
 2. 操作之间添加适当延迟，模拟人类操作
 3. 默认添加适当操作延迟，可配置
- **异常处理：**
 - 坐标越界时抛出异常
 - 操作失败时记录错误日志

4.2.2 键盘操作功能

- **功能描述：**实现键盘的基本操作控制
- **支持操作：**
 1. 输入文本: `type(text)`
 2. 按下单个键: `press(key)`
 3. 释放单个键: `release(key)`
 4. 组合键: `hotkey(*keys)`, 如`hotkey('ctrl', 'c')`
 5. 滚动屏幕: `scroll(direction, steps)`, `direction`为'`up`'或'`down`'
- **输入参数：**操作类型与对应参数
- **输出：**无
- **业务逻辑：**
 1. 支持中英文输入
 2. 支持常见快捷键
 3. 输入速度模拟人类操作
- **异常处理：**
 - 无效按键时抛出异常
 - 输入失败时记录错误日志

4.2.3 操作异常捕获功能

- **功能描述：**捕获并处理操作执行过程中的异常
- **输入参数：**操作函数与参数
- **输出：**操作是否成功
- **业务逻辑：**
 1. 捕获权限不足、窗口未激活等常见异常
 2. 异常发生时记录详细日志
 3. 返回操作结果状态
- **异常处理：**
 - 所有异常均不向上抛出，统一返回布尔值

4.3 Agent核心模块

4.3.1 多模态模型调用接口

- **功能描述：**统一封装通义千问多模态模型的调用，支持本地部署与在线API两种模式
- **输入参数：**
 - `image`: PIL.Image对象（屏幕截图）
 - `prompt`: 用户指令与系统提示词
 - `mode`: 调用模式, '`local`'或'`api`', 默认'`local`'
- **输出：**模型生成的文本响应
- **业务逻辑：**
 1. 本地模式使用Transformers库加载Qwen2-VL-2B-Instruct模型

2. API模式调用通义千问开放平台接口
3. 自动处理模型输入输出格式转换
4. 支持4位量化加载，降低显存占用

- **异常处理：**
 - 模型加载失败时自动切换到API模式
 - API调用失败时重试最多3次
 - 所有异常均记录日志并返回错误信息

4.3.2 系统提示词模板

- **功能描述：**定义引导模型生成正确动作指令的系统提示词
- **提示词内容：**

你是一个桌面GUI操作智能体，请根据当前屏幕截图和用户指令，生成下一步要执行的

请严格按照以下格式输出你的回答，不要添加任何额外内容：

Action：动作类型(参数)

支持的动作类型及参数：

1. click(x=<横坐标>, y=<纵坐标>) - 点击指定坐标
2. type(text="<输入文本>") - 输入指定文本
3. scroll(direction="<up/down>", steps=<步数>) - 滚动屏幕
4. hotkey(key1="<按键1>", key2="<按键2>", ...) - 按下组合键
5. finish(result="<结果描述>") - 任务完成，返回结果

注意事项：

- 每次只输出一个动作
- 坐标必须是整数
- 文本内容用双引号括起来
- 如果任务已经完成，使用finish动作

- **业务逻辑：**
 1. 提示词与用户指令拼接后发送给模型
 2. 可根据实际效果调整提示词内容

4.3.3 动作解析功能

- **功能描述：**解析模型输出的动作指令，转换为可执行的函数调用
- **输入参数：**模型生成的文本响应
- **输出：**动作字典，包含action_type和params
- **业务逻辑：**
 1. 使用正则表达式匹配动作类型与参数
 2. 验证参数的合法性与完整性
 3. 转换参数类型（如字符串转整数）
- **异常处理：**
 - 解析失败时返回None并记录日志

- 不支持的动作类型返回None

4.3.4 任务执行状态管理

- **功能描述：**跟踪任务执行的状态与进度
- **状态定义：**
 - pending: 任务待执行
 - running: 任务执行中
 - success: 任务执行成功
 - failed: 任务执行失败
- **业务逻辑：**
 1. 记录任务开始与结束时间戳
 2. 记录每一步执行的动作与结果
 3. 统计执行步数与重试次数
 4. 任务失败时记录失败原因

4.4 端到端执行模块

4.4.1 流程控制功能

- **功能描述：**协调各模块工作，实现完整的任务执行流程
- **输入参数：**用户自然语言指令
- **输出：**任务执行结果
- **业务逻辑：**
 1. 初始化任务状态
 2. 循环执行以下步骤直到任务完成或达到最大步数：
 - a. 截取当前屏幕
 - b. 调用模型生成动作指令
 - c. 解析动作指令
 - d. 执行动作
 - e. 检查任务是否完成
 3. 返回最终执行结果
- **配置参数：**
 - max_steps: 最大执行步数，默认10
 - retry_count: 单步最大重试次数，默认3

4.4.2 命令行交互界面

- **功能描述：**提供简单的命令行交互方式，接收用户指令并展示执行过程
- **输入参数：**用户通过命令行输入的指令
- **输出：**实时打印执行状态与结果
- **业务逻辑：**
 1. 程序启动时显示欢迎信息与使用说明
 2. 接收用户输入的自然语言指令
 3. 实时打印当前执行步骤与动作
 4. 任务完成后显示结果
 5. 支持输入'exit'退出程序
- **交互示例：**

欢迎使用桌面GUI智能体!

输入'exit'退出程序

请输入指令: 打开Chrome浏览器并访问百度

[步骤1] 执行动作: `click(x=500, y=1000)`

[步骤2] 执行动作: `type(text="www.baidu.com")`

[步骤3] 执行动作: `hotkey(key1="enter")`

任务执行成功: 已打开Chrome浏览器并访问百度首页

4.4.3 日志记录功能

- **功能描述:** 记录系统运行过程中的所有事件与错误
- **日志级别:** DEBUG、INFO、WARNING、ERROR
- **日志内容:**
 - 时间戳
 - 日志级别
 - 模块名称
 - 消息内容
- **业务逻辑:**
 1. 日志同时输出到控制台与文件
 2. 日志文件按周期切割, 保留历史日志
 3. 错误日志单独记录

4.5 系统优化模块

4.5.1 错误检测与自动重试机制

- **功能描述:** 检测操作执行中的错误并自动重试
- **检测的错误类型:**
 1. UI元素未找到
 2. 点击位置无效
 3. 输入失败
 4. 模型调用失败
- **业务逻辑:**
 1. 操作执行后检查是否成功
 2. 如果失败, 等待适当时间后重试
 3. 最多重试指定次数
 4. 重试失败后记录错误并继续下一步

4.5.2 感知模块性能优化

- **功能描述:** 优化屏幕感知模块的速度与准确率
- **优化点:**
 1. 支持指定区域截图, 减少处理范围
 2. 调整OCR识别参数, 提升屏幕文字识别效果

3. 缓存重复出现的UI元素信息

- 业务逻辑：
 - 单帧感知处理耗时 $\leq 300\text{ms}$
 - UI元素识别准确率 $\geq 85\%$

5. 非功能需求

5.1 性能需求

- 单帧屏幕截图耗时 $\leq 50\text{ms}$
- 单帧OCR识别耗时 $\leq 200\text{ms}$
- 本地模型单步推理耗时 $\leq 3\text{秒}$ （4位量化）
- API模式单步推理耗时 $\leq 2\text{秒}$
- 基础任务平均执行时间 $\leq 30\text{秒}$

5.2 兼容性需求

- 支持Windows 10/11、macOS 12+、Ubuntu 20.04+操作系统
- 支持1920x1080、2560x1440等常见分辨率
- 支持100%、125%、150%等常见DPI缩放比例

5.3 稳定性需求

- 基础任务整体成功率 $\geq 70\%$
- 简单任务成功率 $\geq 80\%$
- 系统连续运行无崩溃
- 异常操作不会导致系统死机

5.4 易用性需求

- 命令行界面简洁清晰
- 安装部署步骤 $\leq 5\text{步}$
- 提供详细的使用说明文档
- 错误提示明确易懂

6. 测试方案

6.1 单元测试

- 测试范围：所有独立功能模块
- 测试工具：pytest
- 测试要求：
 - 每个模块编写对应的测试用例

- 核心功能测试覆盖率≥80%
- 所有测试用例执行通过

6.2 集成测试

- **测试范围：**模块之间的接口与交互
- **测试内容：**
 1. 感知模块与Agent模块的数据交互
 2. Agent模块与控制模块的数据交互
 3. 端到端流程的连通性
- **测试要求：**
 - 所有接口调用正常
 - 数据传输格式正确
 - 完整流程能够正常执行

6.3 系统测试

- **测试范围：**整个系统的功能与性能
- **测试任务集：**包含15个不同难度的任务
 - 简单任务（6个）：
 1. 打开计算器并计算1+1
 2. 打开记事本并输入"Hello World"
 3. 调整系统音量
 4. 打开Chrome浏览器
 5. 搜索"Python"关键词
 6. 关闭当前窗口
 - 中等任务（6个）：
 1. 新建Excel表格并输入3行数据
 2. 发送一封简单的邮件
 3. 下载一张图片并保存到桌面
 4. 打开一个本地文档并读取内容
 5. 向聊天软件的指定联系人发送消息
 6. 清空回收站
 - 复杂任务（3个）：
 1. 从浏览器复制一段文字并粘贴到Word文档
 2. 打开PPT并创建一个包含标题和正文的幻灯片
 3. 在文件管理器中查找指定文件并打开
- **评估指标：**
 - 整体任务成功率
 - 不同难度任务的单独成功率
 - 平均任务执行时间
 - 平均错误次数

7. 开发阶段划分

阶段一：技术准备与环境搭建

- 完成技术调研与学习
- 按照本文档3.4节完成开发环境搭建与验证
- 安装所有依赖库并验证功能
- 创建GitHub代码仓库，初始化项目结构

阶段二：核心模块开发

- 开发桌面感知模块（截图、OCR、UI定位）
- 开发桌面控制模块（鼠标、键盘操作）
- 编写单元测试用例
- 完成模块测试与问题修复

阶段三：Agent框架开发

- 实现多模态模型调用接口（本地+API双模式）
- 开发动作解析功能
- 实现任务执行状态管理
- 完成Agent模块测试与功能验证

阶段四：系统集成与基础功能实现

- 集成所有模块，实现端到端执行流程
- 开发命令行交互界面
- 实现基础日志记录功能
- 测试5个基础任务的执行效果并调试

阶段五：系统优化与鲁棒性提升

- 优化系统提示词，提升指令遵循能力
- 实现错误检测与自动重试机制
- 优化感知模块性能与识别准确率
- 解决跨平台兼容性与适配问题

阶段六：系统测试与成果整理

- 设计并执行完整系统测试
- 分析测试结果并修复遗留问题
- 撰写技术文档与使用说明
- 整理项目代码与所有交付物

8. 交付物要求

8.1 代码交付

- 完整的项目源代码，包含所有模块
- 代码添加详细注释，注释覆盖率 $\geq 30\%$
- 符合Python PEP8编码规范
- 提供requirements.txt文件，列出所有依赖
- 提供完整的部署与运行说明

8.2 文档交付

- 项目技术报告
- 系统使用说明文档
- 开发环境配置文档
- 测试报告（包含测试用例与结果）

8.3 演示交付

- 系统演示视频，展示核心功能
- 包含至少3个不同难度任务的执行过程

9. 参考资料

1. AgentScope官方文档: <https://agentscope.readthedocs.io/>
2. 通义千问Qwen2-VL官方文档: <https://huggingface.co/Qwen/Qwen2-VL-2B-Instruct>
3. PaddleOCR官方文档: <https://github.com/PaddlePaddle/PaddleOCR>
4. mss官方文档: <https://python-mss.readthedocs.io/>
5. pynput官方文档: <https://pynput.readthedocs.io/>

10. 附录

10.1 常见问题解决方案

1. **CUDA安装失败**: 参考PyTorch官方文档，安装对应版本的CUDA与cuDNN
2. **模型下载慢**: 使用魔搭社区镜像源下载模型
3. **OCR识别准确率低**: 调整PaddleOCR的识别参数，或使用更高精度的模型
4. **鼠标键盘操作无响应**: 检查程序是否有足够的权限，关闭安全软件的拦截
5. **显存不足**: 使用4位量化加载模型，或切换到API模式

10.2 代码目录结构建议

```
desktop_gui_agent/  
├── README.md  
├── requirements.txt  
├── config.py  
├── main.py  
├── perception/  
│   ├── __init__.py  
│   ├── screenshot.py  
│   ├── ocr_recognizer.py  
│   └── ui_locator.py  
├── control/  
│   ├── __init__.py  
│   ├── mouse_controller.py  
│   └── keyboard_controller.py  
├── agent/  
│   ├── __init__.py  
│   ├── model_client.py  
│   ├── action_parser.py  
│   └── task_manager.py  
├── utils/  
│   ├── __init__.py  
│   ├── logger.py  
│   └── exceptions.py  
└── tests/  
    ├── __init__.py  
    ├── test_perception.py  
    ├── test_control.py  
    └── test_agent.py
```